

RAPPORT DE PROJET DE FIN DE FORMATION

Chatihna.

Chat here, anywhere.

Une application de messagerie premium et minimaliste conçue pour une communication fluide en temps réel, intégrant des fonctionnalités audio avancées et une interface sombre et réactive.

✦ Real-Time Messaging Platform ✦

SPÉCIALITÉ

Développement Digital /
Développement Informatique

ENCADRÉ PAR

Mme Imane Bouziane

RÉALISÉ PAR

Koutaya Sohail

ÉTABLISSEMENT

SOLICODE — Centre de Formation
Numérique

Dédicace

Je dédie ce travail :

À mes chers parents, qui ont toujours été une source de motivation, de soutien et d'encouragement tout au long de mon parcours académique. Leur amour, leurs sacrifices et leurs conseils ont été essentiels à ma réussite.

À ma famille, pour son soutien moral constant et sa confiance en mes capacités.

À mes enseignants et formateurs, qui ont contribué à ma formation et au développement de mes compétences tout au long de cette expérience académique.

À mes amis et collègues, pour leur soutien, leur aide et les moments de partage qui ont rendu cette expérience plus enrichissante.

Enfin, à toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce projet.

Remerciements

Avant de présenter ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont participé à sa réalisation.

Je remercie tout particulièrement l'ensemble du corps pédagogique de l'établissement pour la qualité de la formation dispensée ainsi que pour les connaissances acquises durant mon parcours.

J'adresse mes sincères remerciements à mon encadrant pour son accompagnement, sa disponibilité et ses précieux conseils qui ont grandement contribué à l'aboutissement de ce projet.

Je remercie également les membres du jury qui ont accepté d'évaluer ce travail et de partager leurs remarques et suggestions afin de contribuer à son amélioration.

Mes remerciements vont également à ma famille pour son soutien moral, sa patience et ses encouragements tout au long de cette expérience.

Enfin, je remercie toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce projet.

Résumé

La communication numérique est devenue un élément essentiel dans la vie quotidienne des utilisateurs. Avec l'évolution des technologies web et mobiles, les applications de messagerie instantanée jouent un rôle fondamental dans la facilitation des échanges en temps réel entre les individus.

Dans ce contexte, le présent projet de fin de formation porte sur la conception et le développement d'une application web de messagerie instantanée intitulée « Chatihna ». Cette plateforme a été conçue afin d'offrir aux utilisateurs un environnement moderne, sécurisé et performant leur permettant de communiquer instantanément à travers des conversations privées.

L'application propose plusieurs fonctionnalités telles que l'inscription et l'authentification des utilisateurs, la gestion des profils, l'envoi et la réception de messages texte en temps réel, le partage d'images, l'envoi de messages audio ainsi que l'affichage du statut de connexion et de l'état de lecture des messages.

La réalisation de ce projet s'est appuyée sur des technologies modernes telles que Next.js, React.js, Node.js, Express.js, MongoDB, Socket.IO, Firebase Authentication et Cloudinary. Ces technologies ont permis de développer une application performante répondant aux exigences de rapidité, de sécurité et d'expérience utilisateur.

Les résultats obtenus démontrent que la solution développée répond efficacement aux besoins de communication instantanée tout en garantissant une expérience utilisateur fluide et intuitive.

Mots-clés : Messagerie instantanée, Communication en temps réel, Next.js, React.js, Node.js, MongoDB, Socket.IO, Firebase Authentication, Cloudinary.

Abstract

Digital communication has become an essential part of everyday life. With the rapid evolution of web technologies, instant messaging applications have significantly improved the way people communicate and share information in real time.

This graduation project focuses on the design and development of a web-based instant messaging application called "Chatihna". The platform was developed to provide users with a secure, modern and efficient communication environment.

The application offers several features including user registration and authentication, profile management, real-time text messaging, image sharing, voice messages, online status management and message read indicators.

The project was implemented using modern technologies such as Next.js, React.js, Node.js, Express.js, MongoDB, Socket.IO, Firebase Authentication and Cloudinary. These technologies enabled the development of a reliable and scalable application capable of delivering a high-quality user experience.

The obtained results demonstrate that the developed solution successfully satisfies the identified requirements while providing fast, secure and user-friendly communication services.

Keywords: Instant Messaging, Real-Time Communication, Next.js, React.js, Node.js, MongoDB, Socket.IO, Firebase Authentication, Cloudinary.

Liste des Figures

- Figure 1 : Diagramme de Cas d'Utilisation
- Figure 2 : Diagramme de Classes UML
- Figure 3 : Modèle Conceptuel de Données (MCD)
- Figure 4 : Modèle Logique de Données (MLD)
- Figure 5 : Architecture Générale du Système
- Figure 6 : Logo Visual Studio Code
- Figure 7 : Logo Git et GitHub
- Figure 8 : Logo Next.js
- Figure 9 : Logo React.js
- Figure 10 : Logo Tailwind CSS
- Figure 11 : Logo Node.js
- Figure 12 : Logo Express.js
- Figure 13 : Logo MongoDB
- Figure 14 : Logo Socket.IO
- Figure 15 : Logo Firebase Authentication
- Figure 16 : Logo Cloudinary
- Figure 17 : Architecture du Projet Chatihna
- Figure 18 : Interface de Connexion
- Figure 19 : Interface d'Inscription
- Figure 20 : Interface Principale de Chat
- Figure 21 : Liste des Conversations
- Figure 22 : Partage d'Images
- Figure 23 : Messages Audio
- Figure 24 : Gestion du Profil Utilisateur

Liste des Tableauxⁱ

Tableau 1 : Comparaison des applications de messagerie existantes

Tableau 2 : Besoins fonctionnels du système

Tableau 3 : Besoins non fonctionnels du système

Tableau 4 : Tests du module d'authentification

Tableau 5 : Tests du module de gestion des conversations

Tableau 6 : Tests du module de messagerie

Tableau 7 : Tests du partage d'images

Tableau 8 : Tests des messages audio

Tableau 9 : Tests de la base de données

Table des Matières

INTRODUCTION GÉNÉRALE	8
CHAPITRE 1	9
1.1 Introduction.....	10
1.2 Contexte Général du Projet	10
1.3 Problématique.....	11
1.4 Objectifs du Projet.....	11
1.4.1 Objectif Général.....	11
1.4.2 Objectifs Spécifiques	11
1.5 Étude des Solutions Existantes.....	12
1.5.1 WhatsApp	12
Présentation.....	12
Fonctionnalités Principales	12
Avantages.....	12
Limites	13
1.5.2 Facebook Messenger	13
Présentation.....	13
Fonctionnalités	13
Avantages.....	13
Limites	13
1.5.3 Telegram	13
Présentation.....	13
Fonctionnalités	13
Avantages.....	14
Limites	14
1.6 Comparaison des Solutions Existantes.....	14
1.7 Présentation de la Solution Proposée : Chatihna.....	14
1.8 Planification du Projet.....	15
Phase 1 : Analyse	15
Phase 2 : Conception	15
Phase 3 : Développement.....	15
Phase 4 : Tests	16
Phase 5 : Déploiement	16
1.9 Conclusion.....	16

CHAPITRE 2.....	16
2.1 Introduction.....	17
2.2 Identification des Acteurs.....	17
Utilisateur	17
2.3 Recueil des Besoins.....	18
2.4 Besoins Fonctionnels.....	18
Gestion des Comptes Utilisateurs	18
Gestion des Profils.....	18
Gestion des Conversations.....	18
Gestion des Messages	18
Gestion des Images	19
Gestion des Messages Audio	19
2.5 Besoins Non Fonctionnels.....	19
Performance	19
Sécurité	19
Disponibilité	19
Ergonomie.....	19
Maintenabilité	20
Compatibilité	20
2.6 Tableau des Besoins Fonctionnels	20
2.7 Tableau des Besoins Non Fonctionnels	20
2.8 Diagramme de Cas d'Utilisation.....	21
2.9 Description Textuelle des Cas d'Utilisation	21
Cas d'Utilisation : Inscription	21
Description	21
Préconditions.....	21
Scénario Principal	21
Postconditions.....	21
Cas d'Utilisation : Connexion	21
Description	21
Préconditions.....	21
Scénario Principal	21
Postconditions.....	22
Cas d'Utilisation : Envoi de Message	22
Cas d'Utilisation : Partage d'Image.....	22
2.10 Contraintes du Projet.....	22
Contraintes Techniques	22

Contraintes de Sécurité.....	22
Contraintes de Performance.....	22
2.11 Conclusion.....	23
CHAPITRE 3.....	23
3.1 Introduction.....	24
3.2 Diagramme de Cas d'Utilisation Global.....	24
3.3 Diagramme de Classes	24
Classe User	24
Attributs	24
Méthodes.....	24
Classe Chat	25
Attributs	25
Méthodes.....	25
Classe Message	25
Attributs	25
Méthodes.....	25
3.4 Diagramme de Séquence : Authentification.....	25
3.5 Diagramme de Séquence : Envoi d'un Message	26
3.6 Modèle Conceptuel de Données (MCD).....	26
3.7 Modèle Logique de Données (MLD).....	26
Collection Users.....	26
Collection Chats.....	26
Collection Messages	27
3.8 Architecture Générale du Système.....	27
Couche Frontend.....	27
Couche Backend	27
Base de Données.....	27
Communication Temps Réel	28
Services Externes	28
3.9 Description des Flux du Système.....	28
Flux d'Authentification.....	28
Flux de Messagerie	28
Flux de Partage d'Images.....	28
Flux des Messages Audio	28
3.10 Avantages de l'Architecture Adoptée.....	29
3.11 Conclusion.....	29
CHAPITRE 4.....	29

4.1 Introduction.....	30
4.2 Environnement de Développement	30
4.2.1 Visual Studio Code.....	30
Définition	30
Raison du Choix.....	30
4.2.2 Git et GitHub	30
4.3 Technologies Utilisées	31
4.3.1 Next.js.....	31
4.3.2 React.js.....	31
4.3.3 Tailwind CSS.....	31
4.3.4 Node.js.....	31
4.3.5 Express.js	32
4.3.6 MongoDB	32
4.3.7 Socket.IO	32
4.3.8 Firebase Authentication	32
4.3.9 Cloudinary	33
4.4 Création de la Base de Données.....	33
4.4.1 Collection Users.....	33
4.4.2 Collection Chats.....	33
4.4.3 Collection Messages	34
4.5 Architecture du Projet	34
Structure du Frontend	34
Structure du Backend.....	34
4.6 Réalisation des Interfaces.....	34
4.6.1 Interface de Connexion.....	34
4.6.2 Interface d'Inscription	35
4.7 Module de Messagerie	35
4.7.1 Interface Principale de Chat.....	35
4.7.2 Liste des Conversations	35
4.7.3 Envoi des Messages Texte.....	35
4.7.4 Partage d'Images	35
4.7.5 Messages Audio.....	35
4.7.6 Statut de Connexion.....	36
4.7.7 Statut de lecture des messages	36
4.8 Gestion du Profil Utilisateur	36
4.9 Sécurité de l'Application	36
4.10 Difficultés Rencontrées.....	36

4.11 Conclusion.....	36
CHAPITRE 5.....	37
5.1 Introduction.....	38
5.2 Objectifs des Tests	38
5.3 Types de Tests Réalisés.....	38
5.4 Tests du Module d'Authentification.....	38
5.5 Tests du Module de Profil.....	38
5.6 Tests du Module de Conversations	39
5.7 Tests du Module de Messagerie	39
5.8 Tests du Partage d'Images	39
5.9 Tests des Messages Audio.....	39
5.10 Tests de Performance	40
5.11 Tests de Sécurité.....	40
5.12 Validation du Système	40
5.13 Conclusion.....	40
CHAPITRE 6.....	40
6.1 Introduction.....	41
6.2 Objectifs du Déploiement.....	41
6.3 Préparation de l'Environnement	41
6.4 Déploiement du Frontend.....	41
6.5 Déploiement du Backend	41
6.6 Configuration de MongoDB.....	42
6.7 Configuration Firebase	42
6.8 Configuration Cloudinary	42
6.9 Vérification Après Déploiement	42
6.10 Maintenance du Système.....	42
6.11 Conclusion.....	43
CONCLUSION GÉNÉRALE.....	43

INTRODUCTION GÉNÉRALE

L'évolution rapide des technologies de l'information et de la communication a profondément transformé les habitudes des utilisateurs dans le monde entier. Aujourd'hui, les applications de communication instantanée sont devenues indispensables aussi bien dans la vie personnelle que professionnelle. Elles permettent aux utilisateurs d'échanger rapidement des informations, de partager des contenus multimédias et de collaborer efficacement sans contraintes géographiques.

Face à cette croissance constante des besoins de communication numérique, les plateformes de messagerie instantanée connaissent un développement important. Les utilisateurs recherchent désormais des applications offrant rapidité, sécurité, simplicité d'utilisation et disponibilité permanente.

Dans ce contexte, le projet « Chatihna » a été conçu afin de proposer une solution moderne de communication en temps réel permettant aux utilisateurs d'échanger des messages texte, des images et des messages audio au sein d'une interface intuitive et performante.

La réalisation de ce projet a nécessité plusieurs étapes, allant de l'analyse des besoins jusqu'au déploiement de l'application. Une attention particulière a été accordée à la conception de l'architecture du système, à la gestion des données ainsi qu'à la mise en œuvre des fonctionnalités temps réel grâce à Socket.IO.

L'objectif principal de ce travail est de développer une plateforme capable d'assurer une communication fluide, sécurisée et efficace tout en offrant une expérience utilisateur moderne.

Ce rapport est structuré en six chapitres. Le premier chapitre présente le contexte général du projet ainsi que l'étude des solutions existantes. Le deuxième chapitre est consacré à l'analyse des besoins. Le troisième chapitre présente la conception du système. Le quatrième chapitre décrit la réalisation de l'application. Le cinquième chapitre présente les tests et la validation. Enfin, le sixième chapitre traite du déploiement et de la mise en production avant de conclure par une synthèse générale et les perspectives d'évolution du projet.

CHAPITRE 1

Contexte Général et Étude de l'Existant

1.1 Introduction

L'évolution rapide des technologies de l'information et de la communication a profondément modifié les habitudes de communication des utilisateurs. Grâce à Internet et aux applications web modernes, les échanges d'informations sont devenus plus rapides, plus accessibles et plus efficaces.

Aujourd'hui, les applications de messagerie instantanée constituent l'un des moyens de communication les plus utilisés à travers le monde. Elles permettent aux utilisateurs de communiquer en temps réel, de partager différents types de contenus multimédias et de rester connectés en permanence.

Dans ce contexte, le développement d'une application de messagerie moderne représente un défi important nécessitant la mise en œuvre de technologies adaptées aux systèmes temps réel. Ce chapitre présente le contexte général du projet, la problématique étudiée, les objectifs visés ainsi qu'une analyse des principales solutions existantes sur le marché.

1.2 Contexte Général du Projet

La transformation numérique a engendré une croissance considérable des plateformes de communication en ligne. Les entreprises, les établissements d'enseignement ainsi que les particuliers utilisent quotidiennement des applications de messagerie afin d'échanger des informations instantanément.

Les utilisateurs recherchent désormais des solutions qui offrent :

- Une communication rapide.
- Une interface intuitive.
- Une sécurité renforcée.
- Une disponibilité permanente.
- Une compatibilité avec différents appareils.

Les applications modernes de communication ne se limitent plus à l'échange de messages texte. Elles proposent aujourd'hui des fonctionnalités avancées telles que :

- Le partage d'images.
- Les messages vocaux.
- Les appels audio et vidéo.
- Les notifications en temps réel.
- La gestion des statuts de connexion.

Face à cette évolution, il devient nécessaire de développer des solutions innovantes capables de répondre efficacement aux besoins croissants des utilisateurs.

1.3 Problématique

Malgré l'existence de nombreuses applications de messagerie instantanée, plusieurs défis restent présents.

Parmi les principales difficultés rencontrées :

- La complexité de certaines interfaces utilisateur.
- Les problèmes de confidentialité des données.
- Les limitations liées à certaines fonctionnalités.
- Les performances insuffisantes lors des échanges temps réel.
- Les difficultés d'intégration de nouvelles technologies.

Par ailleurs, le développement d'une application de messagerie constitue une excellente opportunité pour mettre en pratique les connaissances acquises dans le domaine du développement web moderne, des bases de données et des systèmes temps réel.

La problématique principale du projet peut donc être formulée comme suit :

Comment concevoir et développer une application web de messagerie instantanée moderne, sécurisée et performante permettant aux utilisateurs de communiquer efficacement en temps réel ?

1.4 Objectifs du Projet

Le projet Chatihna vise principalement à concevoir et développer une plateforme de communication instantanée répondant aux exigences actuelles des utilisateurs.

1.4.1 Objectif Général

Développer une application web moderne permettant la communication instantanée entre utilisateurs grâce à des échanges sécurisés et synchronisés en temps réel.

1.4.2 Objectifs Spécifiques

Les objectifs spécifiques du projet sont :

- Mettre en place un système d'authentification sécurisé.
- Permettre la création et la gestion des comptes utilisateurs.
- Développer un système de conversations privées.
- Assurer l'échange de messages texte en temps réel.
- Intégrer le partage d'images.
- Intégrer l'envoi de messages audio.
- Afficher les statuts de connexion des utilisateurs.
- Gérer les indicateurs de lecture des messages.
- Garantir la sécurité et la confidentialité des données.
- Offrir une interface moderne et responsive.

1.5 Étude des Solutions Existantes

Avant de développer une nouvelle solution, il est nécessaire d'étudier les plateformes existantes afin d'identifier leurs avantages et leurs limites.

1.5.1 WhatsApp

Présentation

WhatsApp est l'une des applications de messagerie les plus utilisées dans le monde. Développée initialement en 2009 puis acquise par Meta, elle permet la communication instantanée entre utilisateurs.

Fonctionnalités Principales

- Messagerie instantanée.
- Appels audio.
- Appels vidéo.
- Partage de fichiers.
- Discussions de groupe.
- Chiffrement de bout en bout.

Avantages

- Interface simple.

- Sécurité élevée.
- Large communauté d'utilisateurs.
- Excellente stabilité.

Limites

- Dépendance au numéro de téléphone.
- Fonctionnalités limitées pour certaines personnalisations.
- Centralisation des données.

1.5.2 Facebook Messenger

Présentation

Messenger est une application développée par Meta permettant aux utilisateurs Facebook de communiquer instantanément.

Fonctionnalités

- Messagerie instantanée.
- Appels audio et vidéo.
- Partage multimédia.
- Réactions aux messages.
- Discussions de groupe.

Avantages

- Intégration avec Facebook.
- Nombreuses fonctionnalités.
- Large base d'utilisateurs.

Limites

- Consommation importante des ressources.
- Dépendance à l'écosystème Meta.

1.5.3 Telegram

Présentation

Telegram est une application de messagerie basée sur le cloud connue pour sa rapidité et sa flexibilité.

Fonctionnalités

- Messagerie instantanée.
- Partage de fichiers volumineux.
- Canaux de diffusion.
- Bots automatisés.
- Groupes avancés.

Avantages

- Rapidité.
- Grande capacité de stockage.
- Nombreuses options de personnalisation.

Limites

- Certaines fonctionnalités de sécurité ne sont pas activées par défaut.
- Complexité pour les nouveaux utilisateurs.

1.6 Comparaison des Solutions Existantes

Critère	WhatsApp	Messenger	Telegram	Chatihna
Messagerie Temps Réel	Oui	Oui	Oui	Oui
Partage d'Images	Oui	Oui	Oui	Oui
Messages Audio	Oui	Oui	Oui	Oui
Interface Moderne	Oui	Oui	Oui	Oui
Statut de Connexion	Oui	Oui	Oui	Oui
Statut de lecture	Oui	Oui	Oui	Oui
Application Web	Oui	Oui	Oui	Oui
Architecture Moderne	Partielle	Partielle	Partielle	Oui

Tableau 1 : Comparaison des applications de messagerie existantes

1.7 Présentation de la Solution Proposée : Chatihna

Chatihna est une application web de messagerie instantanée conçue pour permettre aux utilisateurs de communiquer efficacement à travers une interface simple et intuitive.

L'application offre :

- L'authentification sécurisée.
- La gestion des profils utilisateurs.

- Les conversations privées.
- Les messages texte en temps réel.
- Le partage d'images.
- Les messages audio.
- Les statuts de connexion.
- Les indicateurs de lecture des messages.

Le système repose sur une architecture moderne basée sur :

- Next.js
- React.js
- Node.js
- Express.js
- MongoDB
- Socket.IO
- Firebase Authentication
- Cloudinary

Cette combinaison technologique permet de garantir de bonnes performances ainsi qu'une excellente expérience utilisateur.

1.8 Planification du Projet

La réalisation du projet Chatihna a été organisée selon plusieurs phases principales.

Phase 1 : Analyse

- L'étude du besoin.
- L'identification des acteurs.
- La définition des fonctionnalités.

Phase 2 : Conception

- La création des diagrammes UML.
- La conception du MCD.
- La conception du MLD.
- L'élaboration de l'architecture générale.

Phase 3 : Développement

- Développer le frontend.
- Développer le backend.
- Intégrer MongoDB.
- Mettre en place Socket.IO.

Phase 4 : Tests

- La validation des fonctionnalités.
- La détection des anomalies.
- L'amélioration des performances.

Phase 5 : Déploiement

- Configurer les services.
- Mettre l'application en production.
- Vérifier son bon fonctionnement.

1.9 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet ainsi que les principales problématiques liées aux applications de messagerie instantanée. Une étude comparative des solutions existantes a permis d'identifier leurs avantages et leurs limites.

Cette analyse a également permis de justifier le développement de la solution Chatihna, conçue pour offrir une communication moderne, rapide et sécurisée grâce aux technologies web les plus récentes.

Le chapitre suivant sera consacré à l'analyse détaillée des besoins fonctionnels et non fonctionnels du système afin de définir précisément les exigences auxquelles devra répondre l'application.

CHAPITRE 2

Analyse des Besoins

2.1 Introduction

Après avoir présenté le contexte général du projet ainsi que les différentes solutions existantes, il est nécessaire de procéder à une analyse détaillée des besoins afin de définir précisément les fonctionnalités que devra offrir l'application Chatihna.

L'analyse des besoins constitue une étape essentielle dans le cycle de développement d'un projet informatique. Elle permet d'identifier les attentes des utilisateurs, de définir les contraintes techniques et de spécifier les fonctionnalités nécessaires au bon fonctionnement du système.

Dans ce chapitre, nous allons présenter les acteurs du système, les besoins fonctionnels et non fonctionnels ainsi que les différents diagrammes UML permettant de modéliser les interactions entre les utilisateurs et l'application.

2.2 Identification des Acteurs

Un acteur représente toute entité externe qui interagit avec le système. Dans le cadre de l'application Chatihna, un seul acteur principal a été identifié.

Utilisateur

L'utilisateur représente toute personne possédant un compte sur la plateforme. Il peut :

- Créer un compte.
- Se connecter à l'application.
- Modifier son profil.
- Rechercher d'autres utilisateurs.
- Créer une conversation.
- Envoyer des messages texte.
- Envoyer des images.
- Envoyer des messages audio.
- Consulter ses conversations.
- Voir les utilisateurs connectés.

- Consulter l'état de lecture des messages.

2.3 Recueil des Besoins

Le recueil des besoins a été effectué à partir de l'étude des applications de messagerie existantes ainsi que de l'identification des fonctionnalités essentielles attendues par les utilisateurs.

2.4 Besoins Fonctionnels

Les besoins fonctionnels décrivent les services que doit fournir le système.

Gestion des Comptes Utilisateurs

- L'inscription d'un nouvel utilisateur.
- La connexion sécurisée.
- La déconnexion.
- La récupération des informations du profil.
- La modification des informations personnelles.

Gestion des Profils

- L'ajout d'une photo de profil.
- La modification des informations personnelles.
- L'affichage des informations utilisateur.
- La consultation du statut de connexion.

Gestion des Conversations

- La création d'une conversation privée.
- L'affichage de la liste des conversations.
- La consultation des discussions existantes.
- La recherche d'utilisateurs.

Gestion des Messages

- L'envoi de messages texte.
- La réception instantanée des messages.
- La consultation de l'historique.
- La suppression éventuelle des messages.

- L'affichage de l'état de lecture.

Gestion des Images

- Le téléversement d'images.
- Le stockage des images.
- L'affichage des images dans les conversations.

Gestion des Messages Audio

- L'enregistrement de messages vocaux.
- L'envoi des fichiers audio.
- La lecture des messages reçus.
- La conservation de l'historique audio.

2.5 Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité du système.

Performance

- Répondre rapidement aux requêtes.
- Assurer une faible latence lors des échanges.
- Supporter plusieurs utilisateurs simultanés.

Sécurité

- Une authentification sécurisée.
- La protection des données personnelles.
- La confidentialité des conversations.
- Le contrôle des accès.

Disponibilité

- Être accessible en permanence.
- Garantir la continuité du service.

Ergonomie

- Posséder une interface intuitive.
- Faciliter la navigation.

- Offrir une bonne expérience utilisateur.

Maintenabilité

- Faciliter la maintenance du code.
- Permettre l'ajout de nouvelles fonctionnalités.
- Respecter une architecture modulaire.

Compatibilité

- Google Chrome.
- Mozilla Firefox.
- Microsoft Edge.
- Safari.

2.6 Tableau des Besoins Fonctionnels

Référence	Fonctionnalité	Description
BF01	Inscription	Création d'un compte utilisateur
BF02	Connexion	Authentification sécurisée
BF03	Profil	Gestion du profil utilisateur
BF04	Conversations	Gestion des discussions privées
BF05	Messages Texte	Envoi et réception des messages
BF06	Images	Partage des images
BF07	Audio	Envoi des messages vocaux
BF08	Temps Réel	Synchronisation instantanée
BF09	Statuts	Gestion Online/Offline
BF10	Statut de lecture	Gestion de l'état Seen

Tableau 2 : Besoins fonctionnels du système

2.7 Tableau des Besoins Non Fonctionnels

Référence	Exigence
BNF01	Performance élevée
BNF02	Sécurité renforcée
BNF03	Disponibilité continue
BNF04	Interface intuitive

Référence	Exigence
BNF05	Compatibilité multi-navigateurs
BNF06	Évolutivité du système
BNF07	Facilité de maintenance

Tableau 3 : Besoins non fonctionnels du système

2.8 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation permet de représenter les interactions entre les acteurs et le système.

Figure 1 : Diagramme de Cas d'Utilisation de Chatihna

(Insérer votre diagramme UML ici)

2.9 Description Textuelle des Cas d'Utilisation

Cas d'Utilisation : Inscription

Description

Permet à un utilisateur de créer un nouveau compte.

Préconditions

- L'utilisateur ne possède pas encore de compte.

Scénario Principal

- L'utilisateur accède à la page d'inscription.
- Il saisit ses informations.
- Le système vérifie les données.
- Le compte est créé.

Postconditions

- Le compte utilisateur est enregistré.

Cas d'Utilisation : Connexion

Description

Permet à un utilisateur authentifié d'accéder à l'application.

Préconditions

- Le compte existe.

Scénario Principal

- L'utilisateur saisit ses identifiants.
- Le système vérifie les informations.
- L'accès est accordé.

Postconditions

- Session utilisateur ouverte.

Cas d'Utilisation : Envoi de Message

Permet à un utilisateur d'envoyer un message à un autre utilisateur.

- Sélection d'une conversation.
- Saisie du message.
- Envoi du message.
- Réception instantanée par le destinataire.

Cas d'Utilisation : Partage d'Image

Permet l'envoi d'images dans les conversations.

- Sélection d'une image.
- Téléversement vers Cloudinary.
- Envoi dans la conversation.

2.10 Contraintes du Projet

Contraintes Techniques

- Utilisation de Next.js.
- Utilisation de React.js.
- Utilisation de Node.js.
- Utilisation de MongoDB.
- Utilisation de Socket.IO.

Contraintes de Sécurité

- Authentification sécurisée.
- Protection des données.
- Contrôle d'accès.

Contraintes de Performance

- Réactivité temps réel.
- Temps de réponse réduit.

2.11 Conclusion

L'analyse des besoins a permis d'identifier l'ensemble des fonctionnalités nécessaires au développement de l'application Chatihna ainsi que les contraintes techniques et non fonctionnelles devant être respectées.

Les besoins recueillis constituent une base essentielle pour la phase de conception qui sera présentée dans le chapitre suivant.

CHAPITRE 3

Conception du Système

3.1 Introduction

Après avoir identifié les besoins fonctionnels et non fonctionnels de l'application Chatihna, il est nécessaire de passer à la phase de conception. Cette étape joue un rôle fondamental dans le cycle de développement puisqu'elle permet de transformer les besoins exprimés en modèles techniques exploitables lors de l'implémentation.

Dans ce chapitre, nous présenterons les différents diagrammes UML, le modèle conceptuel de données (MCD), le modèle logique de données (MLD) ainsi que l'architecture globale de l'application Chatihna.

3.2 Diagramme de Cas d'Utilisation Global

Le diagramme de cas d'utilisation permet de représenter graphiquement les interactions entre les utilisateurs et le système. Dans l'application Chatihna, l'utilisateur constitue l'acteur principal du système.

Figure 2 : Diagramme Global de Cas d'Utilisation (Insérer le diagramme UML ici)

3.3 Diagramme de Classes

Classe User

Attributs

- id
- name
- email
- password
- image
- status
- createdAt
- updatedAt

Méthodes

- register()

- login()
- updateProfile()
- logout()

Classe Chat

Attributs

- id
- userOneId
- userTwoId
- createdAt

Méthodes

- createChat()
- getMessages()

Classe Message

Attributs

- id
- chatId
- senderId
- content
- type
- isSeen
- createdAt

Méthodes

- sendMessage()
- updateSeenStatus()

Figure 3 : Diagramme de Classes UML (Insérer le diagramme UML ici)

3.4 Diagramme de Séquence : Authentification

Le diagramme de séquence permet de représenter l'enchaînement des interactions entre les différents composants du système lors de la connexion.

- L'utilisateur saisit ses identifiants.

- Le frontend envoie la requête.
- Firebase Authentication vérifie les données.
- Le serveur valide l'authentification.
- Le système ouvre la session.
- L'utilisateur accède à son espace personnel.
-

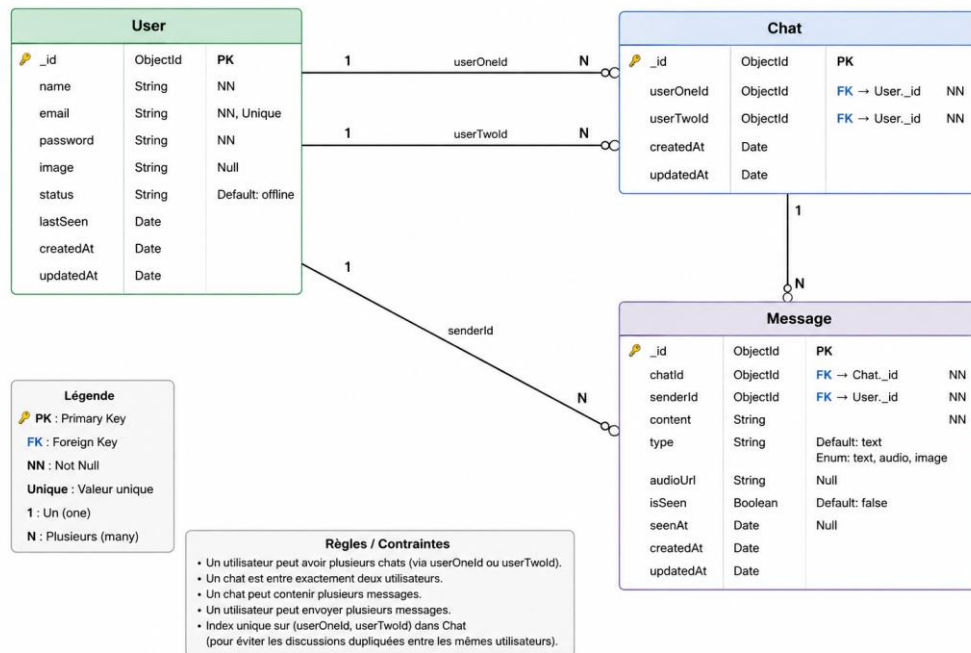
3.5 Diagramme de Séquence : Envoi d'un Message

- L'utilisateur rédige un message.
- Le frontend transmet la requête.
- Le backend reçoit la requête.
- Le message est enregistré dans MongoDB.
- Socket.IO diffuse le message.
- Le destinataire reçoit instantanément le contenu.

3.6 Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données représente la structure générale des informations manipulées par l'application. Le système Chatihna est construit autour de trois entités principales : User, Chat et Message.

Figure 4: Modèle Conceptuel de Données (MCD)



3.7 Modèle Logique de Données (MLD)

Collection Users

- **_id**
- **name**
- **email**
- **password**
- **image**
- **status**
- **lastSeen**
- **createdAt**
- **updatedAt**

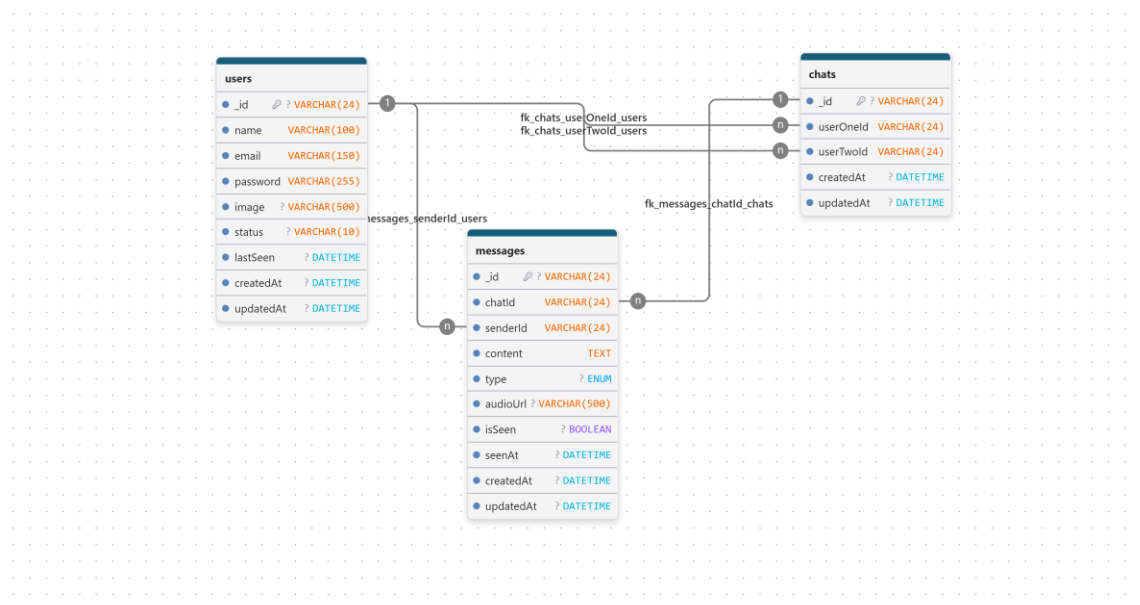
Collection Chats

- **_id**
- **userOneId**
- **userTwoId**
- **createdAt**
- **updatedAt**

Collection Messages

- `_id`
- `chatId`
- `senderId`
- `content`
- `type`
- `audioUrl`
- `isSeen`
- `seenAt`
- `createdAt`
- `updatedAt`

Figure 5: Modèle Logique de Données (MLD)



3.8 Architecture Générale du Système

L'application Chatihna repose sur une architecture client-serveur moderne permettant une séparation claire des responsabilités entre les différents composants.

Couche Frontend

- Next.js
- React.js
- Tailwind CSS

Couche Backend

- Node.js
- Express.js

Base de Données

MongoDB assure le stockage des utilisateurs, des conversations et des messages.

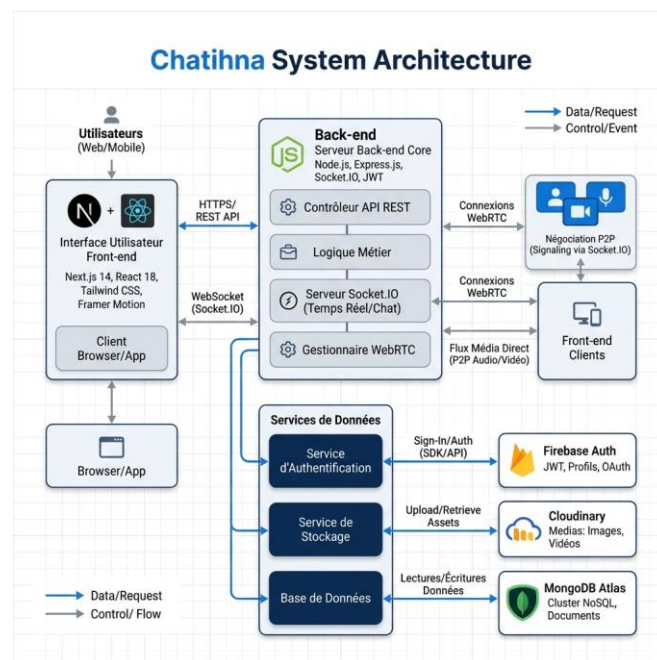
Communication Temps Réel

Socket.IO assure la transmission instantanée des messages, la synchronisation des données et la mise à jour des statuts.

Services Externes

- Firebase Authentication : authentification sécurisée et gestion des sessions.
- Cloudinary : hébergement des images et stockage des médias.

Figure 6: Architecture Générale de Chatihna



3.9 Description des Flux du Système

Flux d'Authentification

- L'utilisateur saisit ses informations.
- Firebase valide les données.
- Le serveur crée la session.
- L'utilisateur accède à son espace.

Flux de Messagerie

- L'utilisateur envoie un message.
- Le backend traite la requête.
- MongoDB enregistre le message.
- Socket.IO transmet les données.
- Le destinataire reçoit le message.

Flux de Partage d'Images

- Sélection d'une image.
- Envoi vers Cloudinary.
- Stockage du lien.
- Affichage dans la conversation.

Flux des Messages Audio

- Enregistrement vocal.
- Création du fichier audio.
- Stockage.
- Transmission au destinataire.

3.10 Avantages de l'Architecture Adoptée

- Modularité : chaque composant est indépendant.
- Performance : Socket.IO améliore la réactivité.
- Évolutivité : ajout facile de nouvelles fonctionnalités.
- Sécurité : Firebase sécurise l'authentification.
- Maintenance : organisation claire du code source.

3.11 Conclusion

La phase de conception a permis de définir l'ensemble des modèles nécessaires à la réalisation de l'application Chatihna. Les différents diagrammes UML, le MCD, le MLD ainsi que l'architecture générale constituent une base solide pour le développement du système.

Le chapitre suivant sera consacré à la réalisation de l'application ainsi qu'à la présentation détaillée des technologies utilisées pour son développement.

CHAPITRE 4

Réalisation

4.1 Introduction

Après les phases d'analyse et de conception, la phase de réalisation consiste à transformer les modèles théoriques élaborés précédemment en une application fonctionnelle répondant aux besoins définis.

L'application Chatihna a été développée en utilisant plusieurs technologies récentes adaptées aux applications web temps réel.

4.2 Environnement de Développement

4.2.1 Visual Studio Code

Définition

Visual Studio Code est un environnement de développement intégré (IDE) développé par Microsoft. Il permet de programmer efficacement grâce à ses nombreuses extensions et fonctionnalités.

Figure 7: Visual Studio Code



Raison du Choix

- Sa légèreté.
- Sa rapidité d'exécution.
- Son support des technologies JavaScript.

- Son intégration avec Git.
- Son large catalogue d'extensions.

4.2.2 Git et GitHub

Git est un système de gestion de versions permettant le suivi des modifications apportées au code source. GitHub est une plateforme collaborative permettant l'hébergement et la sauvegarde des projets.

Figure 8: Logo Git et GitHub



- La gestion des versions.
- Le travail collaboratif.
- La sauvegarde du projet.
- Le suivi des modifications.

4.3 Technologies Utilisées

4.3.1 Next.js

Next.js est un framework React permettant de développer des applications web performantes et optimisées.

Figure 9: Logo Next.js

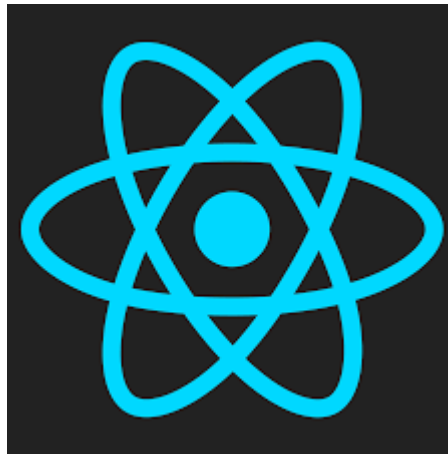


- Son système de routage intégré.
- Son optimisation automatique.
- Son rendu côté serveur.
- Ses performances élevées.
- Son excellente intégration avec React.

4.3.2 React.js

React.js est une bibliothèque JavaScript utilisée pour la création d'interfaces utilisateur interactives.

Figure 10: Logo React.js



- Une architecture basée sur les composants.
- Une excellente réutilisation du code.
- Une interface dynamique.
- Une expérience utilisateur améliorée.

4.3.3 Tailwind CSS

Tailwind CSS est un framework CSS moderne basé sur les classes utilitaires.

Figure 11: Logo Tailwind CSS



- Un développement rapide.
- Une interface moderne.
- Un design responsive.
- Une maintenance simplifiée.

4.3.4 Node.js

Node.js est un environnement d'exécution JavaScript côté serveur.

Figure 12: Logo Node.



- Une gestion rapide des requêtes.
- Une excellente performance.
- Une architecture adaptée aux systèmes temps réel.

4.3.5 Express.js

Express.js est un framework Node.js destiné au développement d'API web.

Figure 13: Logo Express.js



- Simplicité.
- Flexibilité.
- Rapidité de développement.
- Intégration efficace avec MongoDB.

4.3.6 MongoDB

MongoDB est une base de données NoSQL orientée documents.

Figure 14: Logo MongoDB



- Une grande flexibilité.
- Une excellente évolutivité.
- Une bonne performance.
- Une intégration naturelle avec Node.js.

4.3.7 Socket.IO

Socket.IO est une bibliothèque permettant la communication bidirectionnelle en temps réel entre le client et le serveur.

Figure 15: Logo Socket.



- L'envoi instantané des messages.
- La synchronisation des données.
- La mise à jour des statuts en temps réel.
- Une faible latence.

4.3.8 Firebase Authentication

Firebase Authentication est un service d'authentification proposé par Google.

Figure 16: Logo Firebase



- Une authentification sécurisée.
- Une gestion simplifiée des comptes.
- Une intégration rapide.

4.3.9 Cloudinary

Cloudinary est une plateforme spécialisée dans l'hébergement de contenus multimédias.

Figure 17: Logo Cloudinary



- Le stockage sécurisé des images.
- Une gestion simplifiée des médias.
- Une excellente disponibilité.

4.4 Création de la Base de Données

4.4.1 Collection Users

- `_id`
- `name`
- `email`
- `password`
- `image`
- `status`
- `lastSeen`
- `createdAt`
- `updatedAt`

4.4.2 Collection Chats

- `_id`
- `userOneId`
- `userTwoId`
- `createdAt`
- `updatedAt`

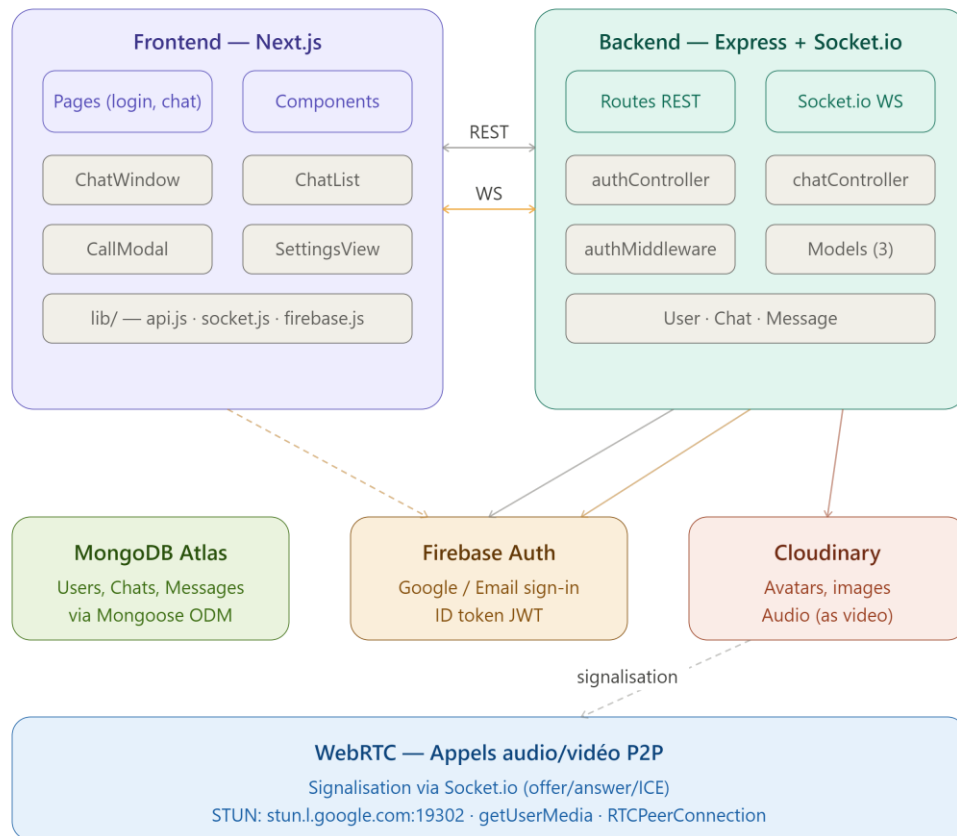
4.4.3 Collection Messages

- `_id`
- `chatId`
- `senderId`
- `content`
- `type`
- `audioUrl`
- `isSeen`
- `seenAt`
- `createdAt`
- `updatedAt`

4.5 Architecture du Projet

L'application adopte une architecture modulaire permettant une séparation claire des responsabilités.

Figure 18 : Architecture du Projet Chatihna



Structure du Frontend

- Les pages Next.js.
- Les composants React.
- Les styles Tailwind CSS.
- Les services API.

Structure du Backend

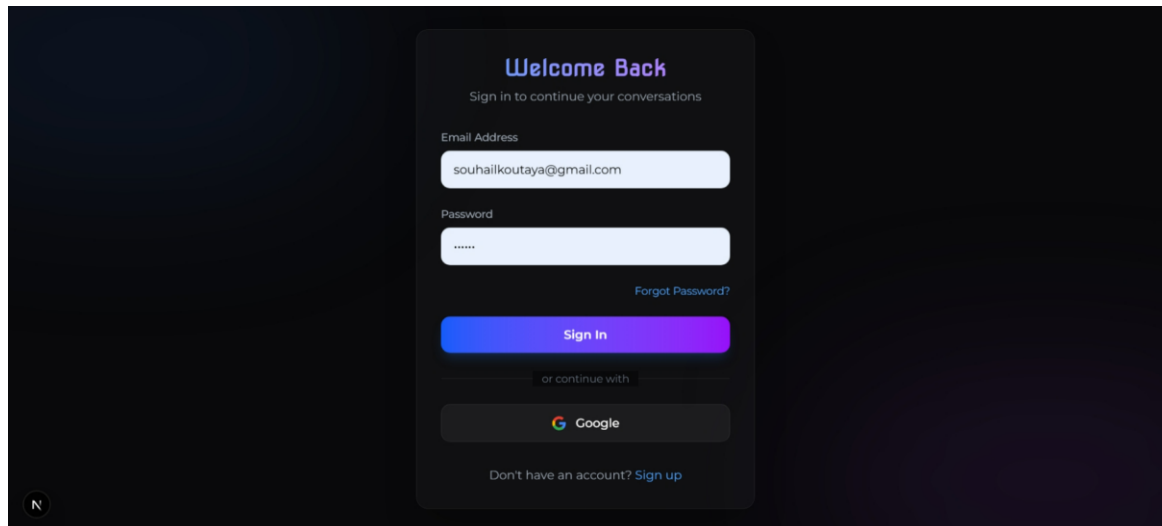
- Les routes API.
- Les contrôleurs.
- Les modèles MongoDB.
- Les services Socket.IO.

4.6 Réalisation des Interfaces

4.6.1 Interface de Connexion

Cette interface permet la saisie de l'email, la saisie du mot de passe et la validation des accès. L'utilisateur doit s'authentifier avant d'accéder à l'application.

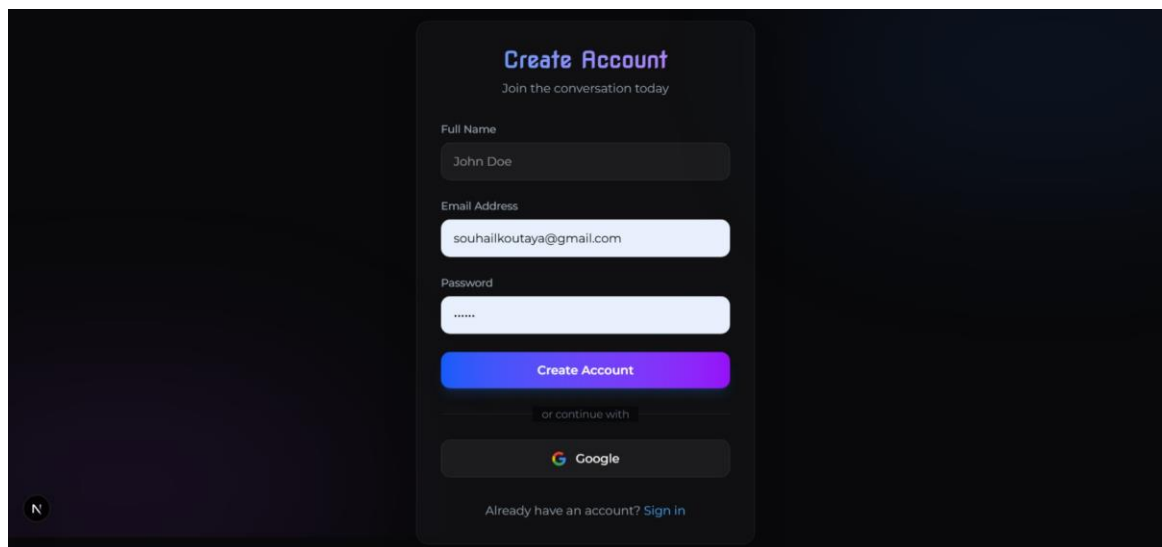
Figure 19: Interface de Connexion

The image shows a login interface titled "Welcome Back" with the subtitle "Sign in to continue your conversations". It features two input fields: "Email Address" containing "souhaikoutaya@gmail.com" and "Password" with masked characters. A "Forgot Password?" link is positioned to the right of the password field. Below these is a prominent blue "Sign In" button. Underneath the button is a separator line with the text "or continue with", followed by a button featuring the Google logo. At the bottom, there is a link that says "Don't have an account? Sign up". A small circular icon with the letter 'N' is visible in the bottom-left corner of the interface.

4.6.2 Interface d'Inscription

Cette interface permet la création d'un compte, la saisie des informations personnelles et l'enregistrement sécurisé des données.

Figure 20: Interface d'Inscription

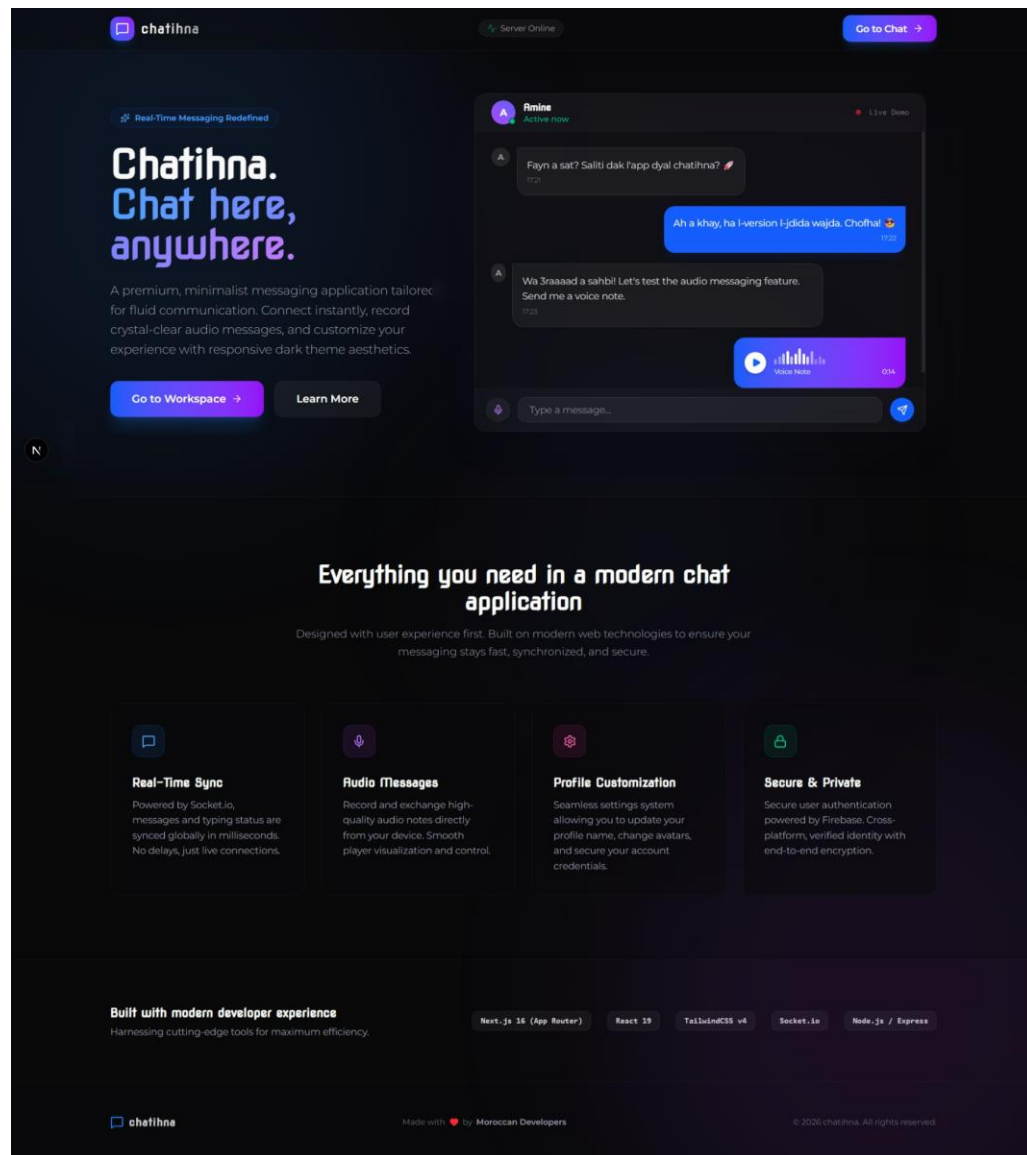
The image displays a registration interface titled "Create Account" with the subtitle "Join the conversation today". It includes three input fields: "Full Name" with "John Doe", "Email Address" with "souhaikoutaya@gmail.com", and "Password" with masked characters. A blue "Create Account" button is located below the password field. Below the button is a separator line with the text "or continue with", followed by a button with the Google logo. At the bottom, there is a link that says "Already have an account? Sign in". A small circular icon with the letter 'N' is visible in the bottom-left corner of the interface.

4.7 Module de Messagerie

4.7.1 Interface Principale de Chat

Cette interface permet la consultation des conversations, la lecture des messages et l'envoi de nouveaux messages.

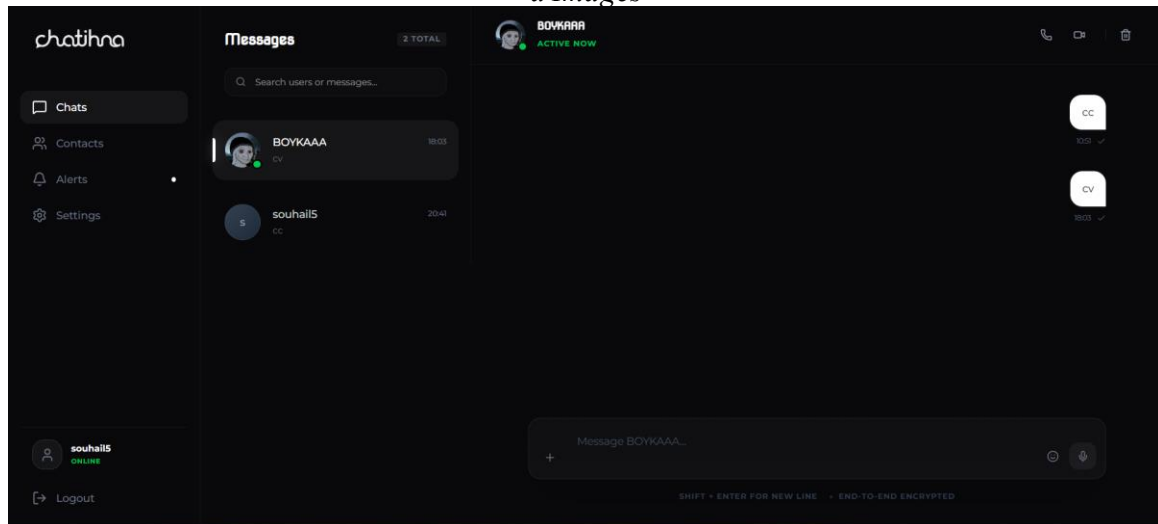
Figure 21: Interface Principale de Chat



4.7.2 Liste des Conversations

- Affichage des discussions.
- Recherche rapide.
- Accès aux conversations.

Figure 22: Liste des Conversations



4.7.3 Envoi des Messages Texte

- La rédaction des messages.
- L'envoi instantané.
- La synchronisation en temps réel.

4.7.4 Partage d'Images

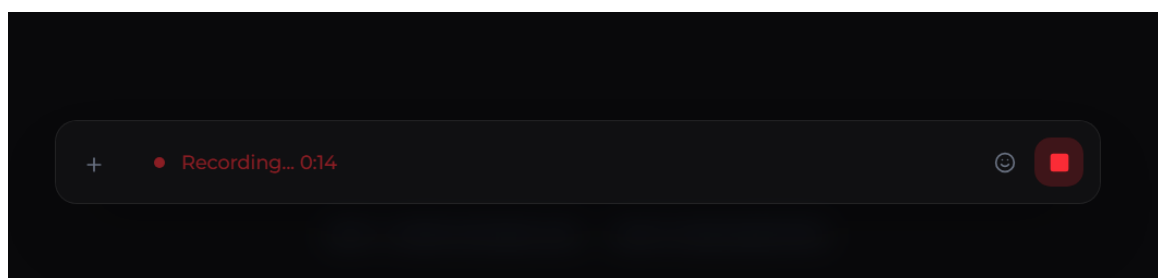
- Sélectionner une image.
- La téléverser vers Cloudinary.
- L'envoyer dans une conversation.

Figure 23: Partage d'Images (Insérer Capture d'écran Images)

4.7.5 Messages Audio

- L'enregistrement vocal.
- L'envoi du message audio.
- La lecture des fichiers reçus.

Figure 24: Messages Audio



4.7.6 Statut de Connexion

- Online.
- Offline.
- Last Seen.

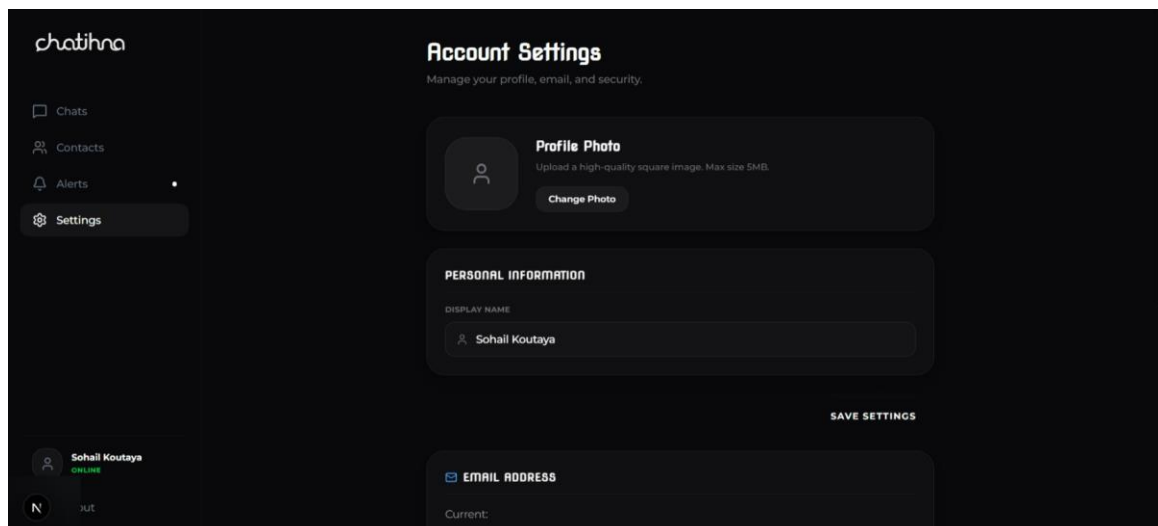
4.7.7 Statut de lecture des messages

Cette fonctionnalité informe l'expéditeur lorsque son message a été consulté.

4.8 Gestion du Profil Utilisateur

- Modifier les informations personnelles.
- Changer la photo de profil.
- Consulter les informations du compte.

Figure 25: Profil Utilisateur



4.9 Sécurité de l'Application

- Authentification : Firebase Authentication sécurise l'accès aux comptes.
- Protection des Données : MongoDB assure le stockage sécurisé des informations.
- Contrôle d'Accès : les fonctionnalités privées sont accessibles uniquement aux utilisateurs authentifiés.
- Sécurisation des Médias : les fichiers sont hébergés sur Cloudinary.

4.10 Difficultés Rencontrées

- Gestion des connexions temps réel.
- Synchronisation des messages.
- Intégration de Socket.IO.

- Téléversement des images.
- Gestion des états de lecture.

Ces difficultés ont été résolues grâce à l'utilisation des technologies adaptées et à une architecture bien structurée.

4.11 Conclusion

La phase de réalisation a permis de développer l'ensemble des fonctionnalités prévues dans le cahier des charges. Grâce aux technologies modernes utilisées, l'application Chatihna offre une communication fluide, rapide et sécurisée.

Dans le chapitre suivant, nous présenterons les différents tests réalisés afin de valider le bon fonctionnement de l'application.

CHAPITRE 5

Tests et Validation

5.1 Introduction

Après la phase de réalisation, il est indispensable de vérifier le bon fonctionnement de l'application développée. Les tests réalisés dans le cadre du projet Chatihna ont pour objectif de vérifier la qualité, la fiabilité, la sécurité ainsi que les performances de l'application.

5.2 Objectifs des Tests

- Vérifier le bon fonctionnement des fonctionnalités.
- Détecter les éventuelles anomalies.
- Valider les exigences fonctionnelles.
- Évaluer la stabilité du système.
- Vérifier les performances de l'application.

5.3 Types de Tests Réalisés

- Tests fonctionnels
- Tests d'intégration
- Tests de validation
- Tests de performance
- Tests de sécurité

5.4 Tests du Module d'Authentification

Test	Action	Résultat Attendu	Résultat Obtenu
T01	Inscription avec données valides	Compte créé	Conforme
T02	Email déjà utilisé	Message d'erreur	Conforme
T03	Mot de passe incorrect	Refus de connexion	Conforme
T04	Connexion valide	Accès autorisé	Conforme

Tableau 4 : Tests du module d'authentification

5.5 Tests du Module de Profil

Test	Action	Résultat
T05	Modification du profil	Réussie
T06	Changement de photo	Réussi
T07	Affichage des informations	Conforme

Tableau 5 : Tests du module de gestion des conversations

5.6 Tests du Module de Conversations

Test	Action	Résultat
T08	Création d'une conversation	Réussie
T09	Recherche d'utilisateur	Réussie
T10	Affichage des conversations	Conforme

Tableau 6 : Tests du module de messagerie

5.7 Tests du Module de Messagerie

Test	Action	Résultat
T11	Envoi de message texte	Réussi
T12	Réception instantanée	Réussie
T13	Historique des messages	Conforme
T14	Mise à jour du statut Vu	Conforme

Tableau 7 : Tests du partage d'images

5.8 Tests du Partage d'Images

Test	Action	Résultat
T15	Téléversement image	Réussi
T16	Affichage image	Conforme
T17	Réception image	Conforme

Tableau 8 : Tests des messages audio

5.9 Tests des Messages Audio

Test	Action	Résultat
T18	Enregistrement audio	Réussi

Test	Action	Résultat
T19	Envoi audio	Réussi
T20	Lecture audio	Conforme

Tableau 9 : Tests de la base de données

5.10 Tests de Performance

- Chargement rapide des pages.
- Réponse fluide des API.
- Synchronisation quasi instantanée.
- Bonne expérience utilisateur.

5.11 Tests de Sécurité

Aucune vulnérabilité critique n'a été détectée lors des tests réalisés.

5.12 Validation du Système

- Toutes les fonctionnalités principales sont opérationnelles.
- Les performances sont satisfaisantes.
- L'interface est ergonomique.
- Les objectifs du projet sont atteints.

5.13 Conclusion

Les différents tests réalisés ont permis de valider le bon fonctionnement de l'application Chatihna. Les résultats obtenus démontrent que le système répond aux besoins fonctionnels et non fonctionnels définis au début du projet.

Le chapitre suivant présentera les différentes étapes de déploiement et de mise en production de l'application.

CHAPITRE 6

Déploiement et Mise en Production

6.1 Introduction

Une fois le développement et les tests terminés, la dernière étape du projet consiste à déployer l'application afin de la rendre accessible aux utilisateurs.

6.2 Objectifs du Déploiement

- Rendre l'application accessible en ligne.
- Assurer la disponibilité du service.
- Garantir la sécurité des données.
- Offrir un environnement stable et performant.

6.3 Préparation de l'Environnement

- Variables d'environnement.
- Connexion à MongoDB.
- Configuration Firebase.
- Configuration Cloudinary.
- Paramètres Socket.IO.

6.4 Déploiement du Frontend

Le frontend développé avec Next.js peut être hébergé sur différentes plateformes. Le choix de Vercel est particulièrement adapté aux projets Next.js grâce à son intégration native.

- Vercel
- Netlify
- Railway

6.5 Déploiement du Backend

- Railway
- Render

- VPS Linux
- DigitalOcean

6.6 Configuration de MongoDB

MongoDB Atlas permet l'hébergement de la base de données avec sauvegarde automatique et haute disponibilité.

- Création du cluster.
- Création de la base de données.
- Configuration des accès.
- Connexion du backend.

6.7 Configuration Firebase

- Gérer les utilisateurs.
- Sécuriser les connexions.
- Contrôler les sessions.

6.8 Configuration Cloudinary

- Le stockage des images.
- L'optimisation des médias.
- La disponibilité des fichiers.

6.9 Vérification Après Déploiement

- Connexion utilisateur.
- Création de conversations.
- Envoi des messages.
- Réception en temps réel.
- Partage d'images.
- Messages audio.

Tous les modules ont été validés avec succès.

6.10 Maintenance du Système

- Mise à jour des dépendances.

- Sauvegarde régulière.
- Surveillance des performances.
- Correction des anomalies.

6.11 Conclusion

Le déploiement de Chatihna a permis de rendre l'application accessible dans un environnement réel tout en garantissant la sécurité, la disponibilité et les performances du système. Cette étape marque l'aboutissement du projet et permet aux utilisateurs de bénéficier de l'ensemble des fonctionnalités développées.

CONCLUSION GÉNÉRALE

Dans le contexte actuel de la transformation numérique et de l'évolution rapide des technologies de communication, les applications de messagerie instantanée occupent une place essentielle dans la vie quotidienne des utilisateurs. Elles permettent d'assurer des échanges rapides, efficaces et accessibles à tout moment.

L'objectif principal de ce projet était de concevoir et développer une application web de messagerie instantanée capable d'offrir un environnement de communication moderne, intuitif et sécurisé. Pour répondre à cet objectif, une étude préalable a été menée afin d'identifier les besoins des utilisateurs et d'analyser les solutions existantes.

La démarche adoptée a permis de mettre en œuvre une solution répondant aux principales exigences fonctionnelles et techniques identifiées. L'application développée permet aux utilisateurs de communiquer en temps réel, d'échanger différents types de contenus et de bénéficier d'une expérience utilisateur fluide grâce à l'utilisation de technologies web modernes.

La réalisation de ce projet a constitué une expérience particulièrement enrichissante, tant sur le plan technique que méthodologique. Elle a permis de consolider les connaissances acquises dans les domaines du développement web, de la conception des systèmes d'information, de la gestion des bases de données ainsi que des architectures de communication temps réel.

Les résultats obtenus démontrent que les objectifs fixés au début du projet ont été atteints avec succès. La solution proposée répond efficacement aux besoins exprimés tout en offrant une base solide pouvant évoluer vers des fonctionnalités plus avancées.

Ce projet représente ainsi une première étape vers le développement d'une plateforme de communication plus complète, capable de s'adapter aux évolutions futures des technologies et aux attentes croissantes des utilisateurs.